

Appunti del corso di Basi di dati

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis

Matteo Drago

Indice

1	Tecnologie per le basi di dati	1
1.1	Architettura transazionale di un DBMS	1
1.2	Transazione	2
1.2.1	Sintassi	2
1.2.2	Proprietà delle transazioni	3
1.3	Architettura di un DBMS	3
2	Linguaggio SQL	6
2.1	Clausola SELECT	7
2.2	Clausola FROM	9
2.3	Clausola WHERE	10
2.3.1	Operatore LIKE	11
2.3.2	Operatore SIMILAR TO	12
2.3.3	Operatore BETWEEN	13
2.3.4	Operatore IN	14
2.3.5	Operatore IS NULL	14
2.4	Clausola DISTINCT	15
2.5	Clausola JOIN	16
2.5.1	Clausola INNER JOIN	17
2.5.2	Clausola LEFT OUTER JOIN	18
2.5.3	Clausola RIGHT OUTER JOIN	19
2.5.4	Clausola FULL OUTER JOIN	19
2.6	Clausola AS	19
2.7	Clausola ORDER BY	20
2.8	Operazioni insiemistiche	22
2.8.1	Operatore UNION	22
2.8.2	Operatore INTERSECT	24
2.8.3	Operatore EXCEPT	25
2.9	Operatori aggregati	25
2.9.1	Operatore COUNT	25
2.9.2	Operatore SUM	26
2.9.3	Operatore MAX/MIN	27
2.9.4	Operatore AVG	27
2.10	Clausola GROUP BY	28
2.10.1	Clausola HAVING	29
2.11	Interrogazioni nidificate	30

2.11.1	Clausola WHERE	30
2.11.2	Clausola EXISTS	31
2.11.3	Clausola SELECT	32
2.11.4	Clausola FROM	32
3	Strutture fisiche e di accesso ai dati	34
3.1	Gestore del buffer	34
3.1.1	Buffer	35
3.1.2	Politica di gestione della memoria	35
3.1.3	Gestione delle pagine	35
3.1.4	Primitive per la gestione del buffer	36
3.1.4.1	Primitiva fix	36
3.1.4.2	Primitiva unfix	36
3.1.4.3	Primitiva setDirty	36
3.1.4.4	Primitiva force	36
3.1.4.5	Primitiva flush	37
3.2	Gestore dell'affidabilità	37
3.2.1	File di LOG	37
3.2.2	Regole di scrittura sul file di LOG	38
3.2.3	Guasti	39
3.3	Gestore dei metodi di accesso	41
3.3.1	Metodi di accesso	41
3.3.2	Organizzazione di una pagina	42
3.3.3	Operazioni sulle pagine	43
3.3.4	Strutture primarie per l'organizzazione dei file	43
3.3.4.1	Struttura sequenziale	43
3.3.4.1.1	Struttura sequenziale disordinata	43
3.3.4.1.2	Struttura sequenziale ad array	44
3.3.4.1.3	Struttura sequenziale ordinata	44
3.3.4.1.4	Indice	45
3.3.4.2	Struttura ad albero	48
3.3.4.2.1	Struttura	48
3.3.4.2.2	Vincoli di riempimento	49
3.3.4.2.3	Operazione: ricerca con chiave K	50
3.3.4.2.4	Operazione: inserimento	51
3.3.4.2.5	Operazione: cancellazione	52
3.3.4.3	Struttura ad accesso calcolato (hash)	53
3.3.4.3.1	Operazione: ricerca con chiave K	54
3.3.4.3.2	Collisioni	54
4	Esecuzione concorrente di transazioni	56
4.1	Tipologie di anomalie	56
4.1.1	Perdita di aggiornamento	57
4.1.2	Lettura inconsistente	57
4.1.3	Lettura sporca	58
4.1.4	Aggiornamento fantasma	58
4.1.5	Inserimento fantasma	59
4.2	Schedule	59
4.2.1	Schedule seriale	60

4.3	Tecniche di gestione della concorrenza	60
4.3.1	View-equivalenza	60
4.3.2	Conflict-equivalenza	65
4.3.3	Locking a due fasi stretto	70
4.3.4	Blocco critico (deadlock)	72
4.3.5	Gestione della concorrenza in SQL	73
5	Ottimizzazione di interrogazioni	74
5.1	Ottimizzazione dipendente dai metodi di accesso	74
5.1.1	Scansione sequenziale	75
5.1.2	Ordinamento	75
5.1.3	Accesso diretto tramite indice	76
5.1.4	Join	77
5.1.4.1	Nested loop join	77
5.1.4.2	Nested loop join con indice	78
5.1.4.3	Block nested loop join	79
5.1.4.4	Merge scan join	80
5.1.4.5	Hash-based join	80
5.2	Scelta del piano esecuzione	81
6	Laboratorio	82
6.1	Comando CREATE TABLE	82
6.1.1	Identificatori	83
6.1.2	Domini elementari	83
6.1.2.1	Tipo carattere	83
6.1.2.2	Tipo boolean	84
6.1.2.3	Tipo numerico esatto	84
6.1.2.4	Tipo numerico approssimato	85
6.1.2.5	Istanti temporali	85
6.1.2.6	Intervalli temporali	85
6.1.3	Domini definiti dall'utente	86
6.1.4	Vincoli intrarelazionali	86
6.1.5	Vincoli interrelazionali	87
6.2	Modifica degli schemi	88
6.2.1	Comando ALTER TABLE	88
6.3	Comando DELETE e DROP TABLE	89
6.4	Comando INSERT INTO e UPDATE	89
6.5	Selezione	90
6.5.1	Clausola SELECT	91
6.5.1.1	Target List	91
6.5.1.2	Concatenazione stringhe	92
6.5.2	Clausola FROM	92
6.5.2.1	Uso di alias	93
6.5.2.2	Target list	93
6.5.3	Clausola WHERE	93
6.5.3.1	Operatore LIKE	93
6.5.3.2	Operatore SIMILAR TO	94
6.5.3.3	Operatore BETWEEN	94
6.5.3.4	Operatore IN	95

	6.5.3.5 Operatore IS NULL	95
	6.5.3.6 Operatore ORDER BY	95
	6.5.3.7 Operatori di aggregazione	96
6.6	Esempi di interrogazioni	97
6.6.1	Interrogazione con raggruppamento: GROUP BY	97
6.6.2	Interrogazione con JOIN	98
6.6.3	Interrogazioni con LEFT OUTER JOIN	99
6.6.4	Interrogazioni con RIGHT OUTER JOIN	99
6.6.5	Interrogazioni con FULL OUTER JOIN	99
6.7	Interrogazioni nidificate e insiemistiche	99
6.7.1	Clausola FROM	100
6.7.2	Clausola WHERE	100
	6.7.2.1 Operatore ANY/SOME	100
	6.7.2.2 Operatore ALL	101
	6.7.2.3 Operatore IN	101
	6.7.2.4 Operatore EXIST	102
6.7.3	Le viste	104
6.8	Indici	105
6.8.1	Creazione indice	106
6.8.2	Caso pratico	107
6.8.3	Tipi di indice	108
	6.8.3.1 B-tree	108
	6.8.3.2 Hash	109
	6.8.3.3 Multi-attributo	109
	6.8.3.4 Indici di espressioni	109
6.8.4	Costo e Pratiche d'uso	110
	6.8.4.1 Explain con due nodi	111
	6.8.4.2 Explain con due nodi – AND con un solo indice	111
	6.8.4.3 Esempi d'uso	111
6.8.5	Explain versione avanzata	113
6.9	Introduzione al controllo di concorrenza in SQL	114
6.9.1	Transazioni	115
	6.9.1.1 Controllo concorrenza funzionamento	115
6.9.2	Livelli di isolamento	117
	6.9.2.1 Sintassi	117
	6.9.2.2 Livello di isolamento: Read Committed	118
	6.9.2.3 Livello di isolamento Repeatable Read	119
	6.9.2.4 Livello di isolamento: Repeatable Read	120
	6.9.2.5 Serializable: avvertenze	122
	6.9.2.6 Cosa fare in caso di failure:	122
6.9.3	Lock Espliciti	122

Ottimizzazione di interrogazioni

Ogni interrogazione sottomessa al DBMS viene espressa in un linguaggio dichiarativo (es. SQL).

E' necessario, quindi, trovare un'equivalente espressione procedurale (es. algebra relazionale) per generare un piano di esecuzione.

L'espressione algebrica va ottimizzata rispetto alle caratteristiche del DBMS a livello fisico e dalla base di dati corrente.

Le fasi di ottimizzazione di una query consistono in:

1. **analisi lessicale e sintattica**: verifica che la query sia scritta correttamente
2. **ottimizzazione algebrica**: si basa sulle regole dell'algebra relazionale
 - anticipo delle selezioni
 - anticipo delle proiezioni
3. **ottimizzazione dipendente dai metodi di accesso**: si basa sulle statistiche e sulle strutture fisiche (profili delle tabelle)
4. **generazione del codice**: si genera un piano di esecuzione per ottenere il risultato della query



Figura 5.1: processo di ottimizzazione di una query

5.1 Ottimizzazione dipendente dai metodi di accesso

Le operazioni tipiche di accesso supportate dal DBMS sono:

- ▷ **scansione delle tuple di una tabella**

- ▷ ordinamento delle tuple
- ▷ accesso diretto tramite indice
- ▷ join con diverse implementazioni

5.1.1 Scansione sequenziale

L'operazione di scansione opera contestualmente altre operazioni:

- ◇ scansione + proiezione (senza eliminazione dei duplicati)
- ◇ scansione + selezione (rispetto a un predicato semplice)
- ◇ scansione + inserimento/cancellazione/modifica

⇒ **costo scansione sequenziale**: $NP(R)$ (numero di pagine della relazione R)

5.1.2 Ordinamento

L'ordinamento viene utilizzato per:

- ◇ ordinare il risultato di un'interrogazione (clausola ORDER BY)
- ◇ eliminare duplicati (clausola SELECT DISTINCT)
- ◇ raggruppare tuple (clausola GROUP BY)

L'ordinamento viene eseguito su memoria secondaria utilizzando l'algoritmo **Z-way-Sort-Merge**:

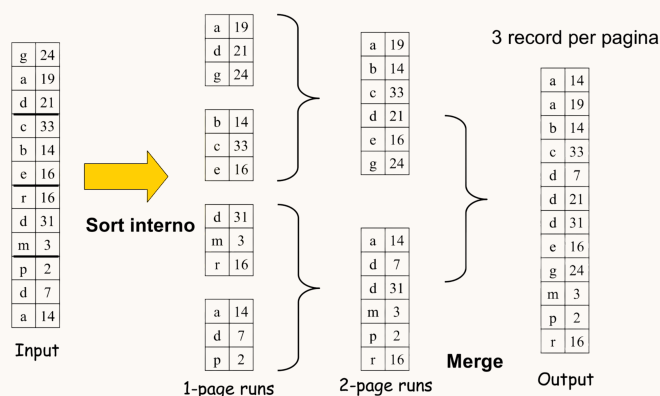
1. **sort interno**:
 - (a) si leggono una alla volta le pagine della tabella
 - (b) le tuple di ogni pagina vengono ordinate con un algoritmo di sort interno (es. quick-sort)
 - (c) ogni pagina ordinata (run) viene scritta su memoria secondaria in un file temporaneo
2. **merge**: vengono unite le run in un'unica run

Esempio: algoritmo Z-way-Sort-merge

Supponiamo di dover ordinare un input che consiste in una tabella di NP pagine e di avere a disposizione NB buffer in memoria centrale ($NB < NP$).

Per semplicità, consideriamo il caso base a due vie ($Z = 2$) e supponiamo di avere solo 3 buffer in memoria centrale ($NB = 3$).

Si hanno 4 pagine composte da 3 tuple ciascuna e si vuole ordinare in base alla prima colonna:

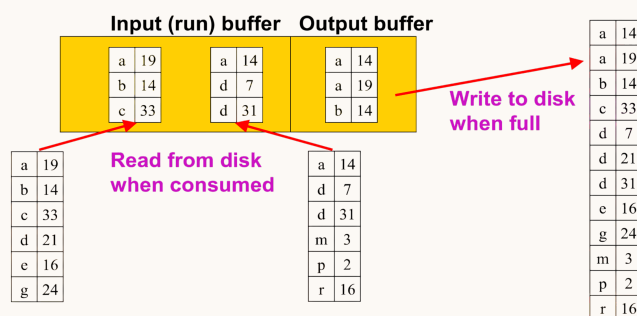


Durante il sort interno si carica e ordina una pagina alla volta ottenendo delle run ordinate.

Si ottengono 4 run ordinate che passano alla fase di merge a due vie ($Z = 2$) e si ottengono 2 run ordinate che passano ad un'ulteriore fase di merge, ottenendo così un'unica run ordinata.

Con $NB = 3$ si hanno solamente 3 buffer a disposizione, quindi, si associano 2 buffer alla run da passare alla fase di merge e 1 buffer per l'output.

Si legge la prima pagina da ciascuna run e si genera la prima pagina dell'output, quando tutti i record di una pagina di run sono stati consumati, si legge un'altra pagina della run:



⇒ **costo ordinamento** (con $Z = 2$ e $NB = 3$):

◇ fase di sort interno: si leggono $NP(R)$ pagine e si scrivono $NP(R)$ pagine

$$2 \cdot NP(R)$$

◇ fase di merge: si leggono $NP(R)$ pagine e si scrivono $NP(R)$ pagine:

$$2 \cdot NP(R)$$

Il numero di passi di merge è:

$$\lceil \log_2(NP(R)) \rceil$$

Il costo complessivo è:

$$2 \times NP(R) \times (\lceil \log_2(NP(R)) \rceil)$$

5.1.3 Accesso diretto tramite indice

Le interrogazioni che fanno uso dell'indice sono:

- ◇ **selezioni con condizione atomica di ugualianza** ($A = v$): richiede indici di hash o B+ tree
- ◇ **selezione con condizione di range** ($A \geq v_1 \wedge A \leq v_2$): richiede indice B+ tree
- ◇ **selezione con condizione costituita da una congiunzione di condizioni di ugualianza** ($A = v_1 \wedge B = v_2$): l'indice viene scelto in base alla condizione più selettiva, l'altra condizione viene verificata filtrando i record restituiti dall'indice
- ◇ **selezioni con condizione costituita da una disgiunzione di condizioni di ugualianza** ($A = v_1 \vee A = v_2$): gli indici possono essere usati in parallelo e poi fare un merge o unione dei risultati

5.1.4 Join

Il join è l'operazione più costosa da eseguire, quindi, l'ottimizzatore è particolarmente attento ad ottimizzare questa operazione, tanto da prevedere diversi algoritmi:

- ▷ **nested loop join**
- ▷ **merge scan join**
- ▷ **hash-based join**

Dal punto di vista logico il join è commutativo, ma dal punto di vista fisico c'è una differenza sostanziale tra $A \bowtie B$ e $B \bowtie A$ che cambia le prestazioni.

5.1.4.1 Nested loop join

Date 2 relazioni in input R e S tra cui è definito il predicato di join PJ, distinguiamo:

- **relazione esterna R**
- **relazione interna S**

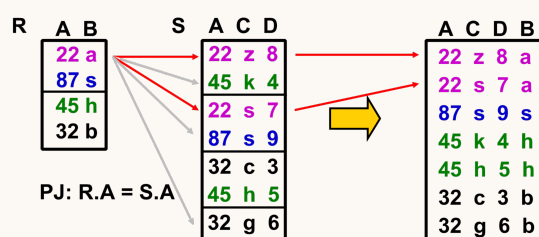
L'algoritmo funziona come segue:

```

1  ∀ tupla tR { --relazione esterna
2      ∀ tupla tS { --relazione interna
3          if (tR, tS) soddisfa PJ
4              aggiungi (tR, tS) al risultato
5      }
6  }
```

Esempio: algoritmo nested loop join

Consideriamo il seguente esempio di join tra le relazioni R e S:



R deve verificare per ogni riga di S che il predicato sia verificato.

⇒ **costo nested loop join** (consideriamo 1 pagina per caricare R e 1 pagina per caricare S):

- ◇ bisogna leggere R completamente una volta
- ◇ per ogni riga di R ($NP(R) = \#RIGHE\ R$) bisogna leggere completamente S

Il costo complessivo è:

$$NP(R) + NR(R) \cdot NP(S)$$

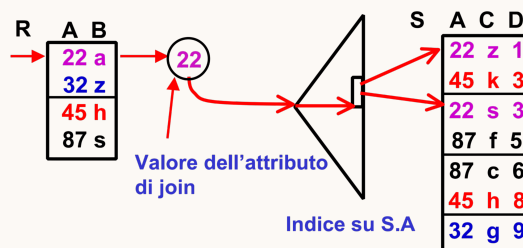
A seconda del numero di righe di R cambia il numero di volte che bisogna leggere $NP(R)$, quindi, la relazione esterna deve essere quella più piccola per minimizzare il costo.

5.1.4.2 Nested loop join con indice

Data una relazione esterna R e una interna S, si ha che la scansione completa di S può essere sostituita dalla scansione basata sull'indice costituito sugli attributi di join di S.

Esempio: algoritmo nested loop join con indice

Consieriamo il seguente esempio di nested loop join con indice tra le relazioni R e S:



L'indice su S.A permette di caricare solo i blocchi di S che contengono l'attributo $A = 22$.

⇒ **costo nested loop join** (usando B+ tree):

$$NP(R) + NR(R) \cdot \left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A, S)} \right)$$

La tabella R deve comunque essere caricata tutta ($NP(R)$) e rimane invariato anche il numero di volte che si esegue il ciclo esterno ($NR(R)$).

Ciò che cambia è il numero di blocchi/pagine di S che si devono caricare tutte le pagine di $\left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A, S)} \right)$.

L'idea è, invece, di caricare tutte le pagine di S si caricano solo quelle che contengono $S.A = R.A$

- ◇ $\frac{NR(S)}{VAL(A, S)}$: rappresenta la stima di quanti blocchi di S contengono A uguale al valore interessato.

Questa stima è anche chiamata **selettività** dell'attributo A.

- ◇ $VAL(A, S)$: numero di valori distinti di A che compaiono in S

Esempio:

Supponiamo che $A = \text{Nome_Regioni}$, quindi, il numero di possibili valori di A è 21.

Consideriamo:

- ◇ $NR(S) = 2100$
- ◇ numero di righe per ogni regione è:

$$\frac{NR(S)}{VAL(A, S)} = \frac{2100}{21} = 1000$$

quindi si ha l'assunzione che il dato sia uniforme, cioè che non siano più o meno lo stesso numero di pagine per ogni regione

- ◇ nel caso pessimo si ha un blocco/pagina per ogni riga

Quindi, il costo del nested loop join con indice è:

$$NP(R) + NR(R) \cdot \left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A, S)} \right)$$

$$NP(R) + NR(R) \cdot (\text{ProfonditàIndice} + 1000)$$

5.1.4.3 Block nested loop join

Il **block nested loop join** è una variante del nested loop in cui ciascuna pagina di S (tabella interna) viene letta per ogni pagina di R (tabella esterna), invece, che per ogni riga di R .

Lo pseudocodice è il seguente:

```

1  ∀ pagina PR di R { --relazione esterna
2      ∀ pagina PS di S { --relazione interna
3          ∀ tupla tR di PR {
4              ∀ tupla tS in PS {
5                  if (tR , tS) soddisfa PJ
6                      aggiungi (tR , tS) al risultato
7              }
8          }
9      }
10 }
```

⇒ **costo block nested loop join**: dipende dallo spazio a disposizione nei buffer.

Supponiamo di avere 1 buffer per R e 1 buffer per S , allora:

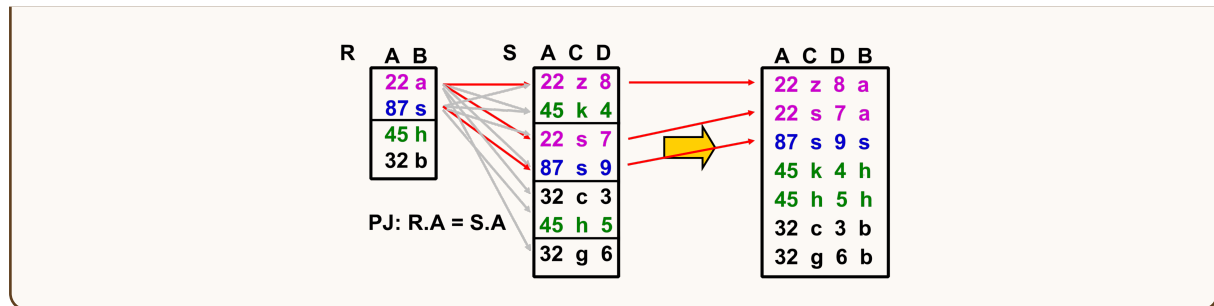
- ◇ bisogna leggere R completamente una volta, quindi $NP(R)$
- ◇ per ogni pagina di R ($NP(R)$) bisogna leggere completamente S quindi $NP(R) NP(S)$

Il costo totale è:

$$NP(R) + NR(R) \cdot NP(S)$$

Esempio: algoritmo block nested loop join

Consideriamo il seguente esempio di block nested loop join tra le relazioni R e S :



5.1.4.4 Merge scan join

Il **merge scan join** è applicabile quando entrambi gli insiemi di tuple in input sono ordinati sugli attributi di join e il join è un equi-join (predicato di join è una condizione di uguaglianza). Ciò accade sia per entrambe le relazioni R e S vale almeno una delle seguenti condizioni:

- relazione è fisicamente ordinata sugli attributi di join (file sequenziale ordinato come struttura fisica)
- esiste un indice sugli attributi di join della tabella che consente una scansione ordinata delle tuple

La logica di questo algoritmo sfrutta il fatto che entrambi gli input sono ordinati per evitare di fare confronti inutili e questo fa sì che il numero di letture totali sia:

$$NP(R) + NP(S)$$

se si accede sequenzialmente alle due relazioni ordinate fisicamente.

- con indici B+ tree su entrambe relazioni il costo può arrivare al massimo (con indici nel buffer) a:

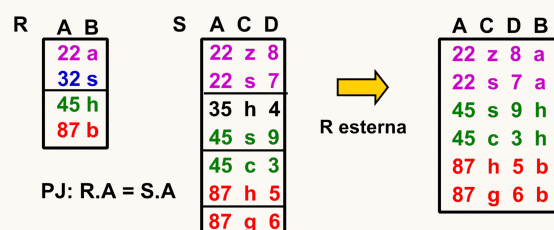
$$NR(R) + NR(S)$$

- con una relazione ordinata e un solo indice il costo è al massimo:

$$NP(R) + NR(S) \text{ (o viceversa)}$$

Esempio: algoritmo merge scan join

Consideriamo il seguente esempio di merge scan join tra le relazioni R e S:



5.1.4.5 Hash-based join

L'algoritmo di **hash-based join** è applicabile solo in caso di equi-join e non richiede né la presenza di indici né input ordinati e risulta particolarmente vantaggioso in caso di relazioni

molto grandi.

L'idea su cui si basa l'hash join è semplice: si suppone di avere a disposizione una funzione hash h che viene applicata agli attributi di join delle due relazioni (es. $R.J$ e $S.J$).

- se t_R e t_S sono 2 tuple di R e S , allora è possibile che sia $t_R.J = t_S.J$ solo se $h(t_R.J) = h(t_S.J)$
- se, viceversa, $h(t_R.J) \neq h(t_S.J)$, allora si ha $t_R.J \neq t_S.J$

Da questa idea si hanno diverse implementazioni, che hanno in comune il fatto che R e S vengono partizionate sulla base dei valori della funzione di hash h , e che la ricerca di matching tuples avviene solo tra partizioni relative allo stesso valore di h .

⇒ **costo hash-based join**:

- costruzione dell'hash map:

$$NP(R) + NP(S)$$

- accesso alle matching tuples che nel caso pessimo può costare:

$$NP(R) \cdot NP(S)$$

5.2 Scelta del piano esecuzione

Data un'espressione ottimizzata in algebra, i passi da eseguire sono:

1. **si generano tutti i possibili piani di esecuzione (alberi) alternativi** (o un sottoinsieme) **ottenuti considerando le seguenti dimensioni**
 - operatori alternativi applicabili per l'accesso ai dati: scan sequenziale o via indice
 - operatori alternativi applicabili nei nodi: nested loop join o hash based join
 - ordine delle operazioni da compiere (associatività)
2. **si valuta con formule approssimate il costo di ogni alternativa in termini di accessi alla memoria secondaria richiesti** (è una stima)
3. **si sceglie l'albero con il costo stimato minimo**

Nella valutazione delle stime di costo si tiene conto del profilo delle relazioni, solitamente memorizzato nel data directory e contenute per ogni relazione T :

- stima della cardinalità ($\#$ tuple): $CARD(T)$
- stima della dimensione di una tupla: $SIZE(T)$
- stima della dimensione di ciascun attributo A : $VAL(A, T)$
- stima del numero di valori distinti per ogni attributo A : $MAX(A, T)$ e $MIN(A, T)$

Esercizio

Consideriamo la seguente base di dati:

```
INGREDIENTE(codice, nome, calorie)
COMPOSIZIONE(ricetta, ingrediente, quantità)
RICETTA(codiceRicetta, nome, regione, tempoPreparazione)
```

e i seguenti vincoli d'integrità:

Composizione.ricetta → Ricetta
Composizione.ingrediente → Ingrediente

Data la seguente query:

```
1 SELECT R.codiceRicetta, I.nome, I.calorie
2 FROM RICETTA R
3 JOIN Composizione C ON R.codiceRicetta = C.ricetta
4 JOIN Ingrediente I ON C.Ingrediente = I.codice
5 WHERE R.regione = 'Veneto';
```

Calcolare il costo dell'interrogazione in termini di numero di accessi alla memoria secondaria con le seguenti ipotesi:

- la selezione (R.Veneto = 'Veneto') su Ricetta richiede una scansione sequenziale della tabella Ricetta.
Inoltre, la selezione rimane nel buffer.
- l'ordine di esecuzione del join è:

(Ricetta ⋈ Composizione) ⋈ Ingrediente

- le operazioni di join vengono eseguite con la tecnica nested loop join con 1 pagina di buffer a disposizione per ogni tabella
- il risultato del primo join (Ricetta ⋈ Composizione) viene interamente mantenuto nel buffer
- il numero di pagine e righe per ogni tabella è il seguente:

Tabella	NP	NR	VAL(regione)	VAL(Ricetta)	VAL(Qtà)
Ingrediente	30	270			
Composizione	780	19800		90	50
Ricetta	150	900	18		

Svolgimento

1. selezione su ricetta:
2. join su Composizione:
3. join su Ingrediente: